ITA06 - MACHINE LEARNING

ASSIGNMENT 2 – CO2

Name: L.MEGHANA

Reg No: 192524107

CO2 Coverage: Neural network representation; perceptrons and perceptron learning algorithm; multilayer perceptrons; introduction to deep neural networks; backpropagation; overfitting and regularization; hyperparameter tuning; optimization strategies.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: 02-11-26

Q1 – Design a Neural Network Representation for a Classification Problem 6 Marks

Problem Context – Manufacturing Quality Inspection

A manufacturing unit records three measurements for each component: Surface Roughness | Dimensional Deviation | Vibration Level. Each component is labelled Acceptable (0) or Defective (1). The unit proposes a neural network to support quality inspection.

Your Challenge

Design a neural network representation for this binary classification problem. Determine:

- The input features and their numerical representation.
- Any preprocessing required and why it is necessary.
- Two candidate architectures: one without a hidden layer and one with a hidden layer.
- The number of input and output neurons, suitable activation functions and an output decision rule.
- The number of trainable weights and biases in each architecture.

Draw both architectures and formulate the mathematical computation from input to output. For one architecture, assign small numerical values to the inputs, weights and biases. Manually calculate the forward pass and resulting classification.

Design Inquiry

The inspection team suspects that a component becomes defective only when certain measurements interact. Explain how this requirement influences your architecture choice and identify what evidence would be needed to justify using a hidden layer.

Evidence Expected

Two labelled network diagrams, justified representation and activation choices, correct parameter counts, a worked forward pass and a reasoned architecture recommendation.

Q2 – Investigate the Learning Behaviour and Limitations of a Perceptron 4 Marks

Problem Context – Warehouse Dispatch Decisions

A warehouse uses two binary indicators: Packaging Complete | Quality Check Passed. Initially, dispatch is permitted only when both indicators are 1.

Your Challenge

Implement the perceptron learning algorithm for this decision. Before implementation:

- Construct the complete input–output table.
- Predict whether a perceptron can learn the decision.
- Support your prediction using a geometric representation.

Specify your initial weights, bias, learning rate, classification threshold and maximum number of epochs. Record predictions before training, weight and bias updates during the first epoch, misclassifications per epoch and final predictions.

Changed Requirement

For a separate review flag, the warehouse now wants an output of 1 when exactly one of the two indicators is 1. Train the same perceptron on this new requirement using at least two learning rates. Determine whether changing the learning rate resolves the difficulty. Distinguish between a training configuration problem and a limitation of the model representation.

Evidence Expected

Both truth tables, geometric reasoning, perceptron implementation, update trace, training records and an evidence-based explanation of the changed requirement.

Q3 – Develop and Verify Backpropagation for a Multilayer Perceptron 6 Marks

Problem Context – Learning an Exclusive Decision Rule

The warehouse review flag in Q2 requires a nonlinear decision boundary. Investigate a multilayer perceptron for this task.

Your Challenge

Develop a network containing: 2 Input Neurons → 2 Hidden Neurons → 1 Output Neuron. Use sigmoid activation functions and a clearly defined binary cross-entropy loss. Before implementation:

- Write the forward-pass equations.
- Derive the loss gradients for the output and hidden layers.
- State the weight and bias update equations.
- Explain why the hidden-layer activation must introduce nonlinearity.

Using stated initial parameters and one training example, manually calculate the hiddenlayer outputs, predicted output and loss, gradients, and updated weights and biases after one training step.

Implement forward propagation and backpropagation using Python and NumPy. An automatic differentiation library must not replace the gradient computation assessed in this question.

Validation Inquiry

Check selected analytical gradients using numerical finite differences. Train the network on all four input combinations. Plot loss across epochs and report the final outputs. If training does not succeed, investigate the result using evidence rather than reporting only the final accuracy.

Evidence Expected

Gradient derivation, a worked numerical update, implementation, numerical gradient checks, loss curve and interpretation of the final predictions.

Q4 – Investigate Whether a Deeper Network Is a Better Design **6 Marks**

Problem Context – Classification with Nonlinear Feature Relationships

A monitoring system must classify observations whose features have nonlinear relationships. Use a reproducible synthetic binary classification dataset containing 1,000 observations and two numerical features, such as two interleaving crescent-shaped classes.

Your Challenge

Compare a network with one hidden layer and a deep network with three hidden layers. Choose and justify hidden-layer widths. Report the parameter count of each model.

Before training, predict how the architectures may differ in their ability to represent the classification boundary, training behaviour, generalization and computational effort.

Use the same training, validation and test partitions for both networks. Keep other training settings consistent and use at least three random seeds. Record:

- Training and validation loss.
- Training and validation accuracy.
- Final test accuracy.
- Parameter count and training time.

Visualise the decision boundaries. Reserve the test set for final evaluation after architecture selection using validation data.

Design Inquiry

Recommend an architecture for this problem. Explain whether any observed improvement can reasonably be attributed to depth alone, considering differences in parameter count and training behaviour.

Evidence Expected

Justified architectures, predictions before training, reproducible comparisons, learning curves, decision boundaries and an evidence-based recommendation.

Q5 – Diagnose Overfitting and Evaluate Regularization **7 Marks**

Problem Context – Learning from Limited Labelled Data

The monitoring system has only 100 labelled observations available for training, although separate validation and test data remain available. A comparatively large neural network is proposed.

Your Challenge

Using the dataset from Q4, construct this limited-data experiment and train an unregularized baseline. Before training, predict the likely relationship between training and validation performance.

Investigate the baseline using training and validation loss curves, accuracy, and the gap between training and validation performance. Compare it with three separate interventions:

- L2 regularization.
- Dropout.
- Early stopping.

Keep architecture, data partitions and other settings consistent wherever applicable. State the regularization strength, dropout rate and early-stopping rule.

Analysis Inquiry

Determine:

- Whether the baseline shows evidence of overfitting.
- Which intervention provides the most useful improvement.
- Whether any setting introduces underfitting.
- How each intervention changes learning behaviour.
- Whether higher training accuracy necessarily indicates a better model.

Select the preferred configuration using validation evidence and evaluate it on the test set only after selection.

Evidence Expected

A controlled limited-data experiment, baseline diagnosis, three regularization comparisons, learning curves, justified configuration selection and final test results.

Q6 – Engineer a Hyperparameter and Optimization Strategy **8 Marks**

Problem Context – Training Under a Computational Budget

The monitoring team must select an effective training configuration while limiting experimentation to 12 candidate training runs.

Your Challenge

Using the dataset and fixed partitions from Q4, design a search plan investigating learning rate, batch size, hidden-layer width and optimization strategy. Include SGD, SGD with momentum and Adam.

Before experimentation:

- Define the candidate configurations.
- Justify how the 12-run budget is allocated.
- State which comparisons isolate the effect of one variable.
- Predict the behaviour of a learning rate that is too small or too large.
- Define the validation measure used to select the final configuration.

Use a consistent maximum epoch budget and stopping rule. Fit preprocessing only on the training data. Record configuration details, best validation performance, epoch of the best validation result, training time and signs of unstable or unsuccessful learning.

Selection Inquiry

Recommend a final configuration using validation results. Explain:

- Which factors had the strongest observed influence.
- The trade-off between performance and training cost.
- Which conclusions are limited by the small search budget.
- Why the test set must remain outside the tuning process.

After selecting the configuration, report its test performance. Do not revise the configuration based on the test result.

Evidence Expected

A justified 12-run search plan, predictions, configuration–result table, controlled comparisons, optimization analysis and a validation-based final selection.

DAY-4 DESIGN CHALLENGE

Q7 – Diagnose, Redesign and Validate a Neural Network Solution 13 Marks

Review the evidence generated in Q2–Q6. Select ONE solution exhibiting a significant limitation, such as failure to represent the target decision, overfitting, slow convergence, unstable training, excessive model size or training cost.

Your redesign must address a diagnosed limitation. Simply replacing the model with a larger network or a different optimizer is insufficient without technical justification.

A. Diagnose the Learning or Computational Bottleneck 2 Marks

Identify the limitation using your original results. Determine whether it primarily arises from model representation, limited data, gradient behaviour, training configuration or computational cost.

Evidence Expected

A precise diagnosis supported by learning curves, prediction errors, gradient observations or performance measurements.

B. Define the Design Objective **2 Marks**

State a measurable redesign objective derived from the diagnosis. Examples include reducing the generalization gap, reaching specified validation performance in fewer epochs, or reducing parameter count while retaining acceptable performance. Specify the baseline, target and acceptable trade-offs before redesign.

Evidence Expected

A measurable objective directly linked to the identified limitation.

C. Design the Revised Learning Strategy **4 Marks**

Develop a revised solution using the topics covered in this assignment. Explain what changes and why, which property of the original problem it addresses, the revised architecture or training procedure, any additional computation or memory required, and new limitations or trade-offs. Provide a network diagram, pseudocode or training workflow.

Evidence Expected

A coherent redesign, appropriate design representation and technical justification connecting each change to the diagnosis.

D. Predict Before Implementing **1 Marks**

Predict how the redesigned solution should behave and identify the measure expected to improve.

Evidence Expected

A clear, testable prediction recorded before validation.

E. Implement and Validate the Redesign **2 Marks**

Implement a prototype and compare it with the original solution using the same data partitions and comparable evaluation conditions. Use measures appropriate to your objective, such as validation loss, accuracy, epochs to convergence, parameter count or training time.

Evidence Expected

Reproducible comparative results and relevant tables or plots.

F. Justify the Final Design **2 Marks** Evaluate whether the redesign achieved its objective. Discuss the prediction, observed improvement, new costs and conditions under which the original solution may remain preferable. A redesign that does not improve performance can still receive credit when the failed hypothesis is correctly investigated and explained.

Evidence Expected

A critical evaluation supported by results and a justified final design decision.

MARK DISTRIBUTION

Question	Assessed Area	Marks
Q1	Neural network representation and forward computation	6
Q2	Perceptron learning and representational limitations	4
Q3	Multilayer perceptron and backpropagation	6
Q4	Deep networks and architecture evaluation	6
Q5	Overfitting and regularization	7
Q6	Hyperparameter tuning and optimization	8
Q7	Day-4 Design Challenge – Diagnose, Redesign and Validate	13
TOTAL		50

Q1 – DESIGN A NEURAL NETWORK REPRESENTATION FOR A CLASSIFICATION PROBLEM (6 MARKS)

1.1 Problem and features

The objective is to classify a manufactured component as Acceptable (0) or Defective (1) from three sensor measurements. This is binary supervised classification.

Feature	Symbol	Representation	Purpose
Surface Roughness	x	Real-valued	Represents the component surface finish.
Dimensional Deviation	x	Real-valued	Measures deviation from the target dimension.
Vibration Level	x	Real-valued	Represents vibration signature during operation.
Target	y	0 / 1	0 = Acceptable; 1 = Defective.

1.2 Preprocessing

- Inspect missing values, duplicate rows, sensor faults and outliers; resolve using documented engineering rules.
- Standardize each feature: $z_i=(x_i-\mu_i)/\sigma_i$. This makes measurements with different scales comparable and often improves gradient-based training.
- Estimate μ and σ from training data only. Apply the same values to validation, test and production data to avoid data leakage.
- Keep the raw measurements for auditability. Use robust scaling if extreme sensor outliers are frequent.

1.3 Architecture A – No hidden layer

Diagram: x ─┐ x ─┼──> [One output neuron: sigmoid] ──> p(defective) ──> threshold 0.5 x ─┘ ↑ bias b Computation:

$z=w\ x\ +w\ x\ +w\ x\ +b$; $p=\sigma(z)=1/(1+e^{z})$. Predict Defective when $p\geq 0.5$

(equivalently $z \geq 0$); otherwise predict Acceptable. Sigmoid is appropriate for a binary output.

Parameter count = 3 weights + 1 bias = 4 trainable parameters.

1.4 Architecture B – One hidden layer

Diagram: [x ,x ,x] → [4 hidden neurons, ReLU] → [1 output neuron, sigmoid] → predicted class.

Hidden computation: a =ReLU($\Sigma_i w_i$ x_i+b), j=1,…,4. Output: $z_o = \Sigma$ v a $+b_o$; $p = \sigma(z_o)$.

Parameter count: input-to-hidden weights 3×4=12; hidden biases 4; hidden-to-output weights 4; output bias 1. Total = 12+4+4+1 = 21 trainable parameters.

1.5 Manual forward pass – Architecture A

Given standardized input x=[0.5,−1.0,1.5], weights w=[0.8,−0.4,0.3] and bias b=−0.2:

$z = (0.8)(0.5)+(-0.4)(-1.0)+(0.3)(1.5)-0.2 = 0.4+0.4+0.45-0.2 = 1.05$.

$p = \sigma(1.05) = 1/(1+e^{-1\cdot})$)≈0.741. Since $p \geq 0.5$, predicted class = 1 (Defective). These are illustrative parameters, not fitted weights.

1.6 Architecture choice and design inquiry

Recommend the hidden-layer model when defect status depends on interactions among the three measurements. A single output neuron produces a linear decision boundary; hidden ReLU neurons combine features nonlinearly. The hidden layer is justified if held-out data demonstrates consistent improvement in defect recall/F1 or validation loss, across several random seeds, and if interaction patterns/domain knowledge support it. A fair comparison should consider parameter count, false-negative cost, training stability and generalization—not training accuracy alone.

Q2 – INVESTIGATE PERCEPTRON LEARNING BEHAVIOUR AND LIMITATIONS (4 MARKS)

2.1 Dispatch decision: AND

Packaging Complete x	Quality Check Passed x	Dispatch target y
0	0	0
0	1	0
1	0	0
1	1	1

AND is linearly separable: the point (1,1) is positive, while (0,0), (0,1), (1,0) are negative. A straight line can separate these regions, so a perceptron can learn this decision.

2.2 Perceptron setup and first-epoch trace

Set initial weights w=[0.2,0.2], bias b=−0.3, learning rate η=0.1, threshold θ=0.5 and maximum epochs=20. Prediction $\hat{y}=1$ if net=$w \cdot x+b \geq \theta$, else 0. Update: $w_i \leftarrow w_i+\eta(y-\hat{y})x_i$; $b \leftarrow b+\eta(y-\hat{y})$.

x	y	Initial net	Initial $\hat{y}$	Error $y-\hat{y}$	Update
(0,0)	0	−0.3	0	0	None
(0,1)	0	−0.1	0	0	None
(1,0)	0	−0.1	0	0	None
(1,1)	1	0.1	0	1	w=[0.3,0.3], b=−0.2

After epoch 1, only (1,1) is wrong. At epoch 2 its net is 0.4, still below 0.5; another update gives w=[0.4,0.4], b=−0.1, making net(1,1)=0.7 and prediction 1. The other inputs remain below threshold. Subsequent epochs have zero errors. Thus the AND target is learned.

2.3 Changed requirement: XOR (exactly one indicator)

x	x	Review flag y=XOR
0	0	0
0	1	1
1	0	1
1	1	0

XOR is not linearly separable. Positive samples (0,1) and (1,0) occupy opposite corners, and negatives (0,0),(1,1) occupy the other corners. No single straight line can separate the positive and negative points. A single perceptron cannot represent XOR.

Learning rates η=0.1 and η=0.01 may change the update size, convergence speed or oscillation, but neither changes the perceptron's linear hypothesis class. Therefore, changing learning rate does not solve the representational limitation. A nonlinear MLP is required.

2.4 Algorithm

```
for epoch in range(max_epochs):
    errors = 0
    for x, y in rows:
```

```
        y_hat = 1 if dot(w, x) + b >= threshold else
0       delta = learning_rate * (y - y_hat)       w =
w + delta * x       b = b + delta       errors +=
int(y_hat != y)     record(errors)     if errors == 0:
break
```

Q3 – DEVELOP AND VERIFY BACKPROPAGATION FOR AN MLP (6 MARKS)

3.1 Network and forward propagation

Use 2 inputs → 2 sigmoid hidden neurons → 1 sigmoid output. Let $W \in R^{2x2}$, $b \in R^2$, $w \in R^2$, $b \in R$.

$z = W x + b$; $a = \sigma(z)$; $z = w^T a + b$; $\hat{y} = \sigma(z)$. Binary cross-entropy: $L = -[y \ln(\hat{y}) + (1-y)\ln(1-\hat{y})]$.

3.2 Gradient derivation

With sigmoid output and binary cross-entropy, $\delta = \partial L/\partial z = \hat{y} - y$. Output gradients: $\partial L/\partial w = \delta a$; $\partial L/\partial b = \delta$.

Hidden error: $\delta = (w \delta) \odot a \odot (1-a)$. Hidden gradients: $\partial L/\partial W = \delta x^T$; $\partial L/\partial b = \delta$. Gradient descent update: $\theta_{new} = \theta_{old} - \eta \partial L/\partial \theta$.

A nonlinear hidden activation is necessary: a sequence of linear layers collapses to one linear transformation and cannot represent XOR.

3.3 Worked one-example update

Use $x = [0,1]^T$, $y = 1$, $\eta = 0.1$; $W = [[0.1, 0.2], [-0.1, 0.1]]$, $b = [0,0]$, $w = [0.2, -0.3]$, $b = 0.1$.

$z = [0.2, 0.1]^T$; $a = [\sigma(0.2), \sigma(0.1)]^T \approx [0.549834, 0.524979]$.

$z \approx 0.2(0.549834) - 0.3(0.524979) + 0.1 = 0.052472$. $\hat{y} = \sigma(z) \approx 0.513115$.

$L = -\ln(0.513115) \approx 0.6674$. $\delta = \hat{y} - 1 \approx -0.486885$. Output gradients: $\partial L/\partial w \approx [-0.2677, -0.2555]$, $\partial L/\partial b \approx -0.486885$.

Hidden deltas: $\delta \approx -0.02407$ and $\delta \approx 0.03628$. Since $x = [0,1]$, $\partial W \approx [[0, -0.02407], [0, 0.03628]]$, $\partial b \approx [-0.02407, 0.03628]$.

Updated parameters: $w \approx [0.22677, -0.27445]$, $b \approx 0.148689$; $W \approx [[0.1, 0.202407], [-0.1, 0.096372]]$, $b \approx [0.002407, -0.003628]$.

3.4 Numerical gradient check and XOR validation

For parameter θ, finite-difference gradient is [L(θ+ε)−L(θ−ε)]/(2ε), ε=10 . Compare to analytical gradient using relative error |g_a−g |/max(1,|g_a|,|g |). Small error (often <10) supports correct derivation and implementation.

Train the 2–2–1 network on all four XOR patterns; plot BCE loss by epoch. Final output should approach [0,1,1,0] in the order (0,0),(0,1),(1,0),(1,1). If unsuccessful, inspect learning rate, initialization, gradient signs, nonlinear activation and sufficient epochs. The assessed implementation must use NumPy analytical gradients, not automatic differentiation.

Q4 – INVESTIGATE WHETHER A DEEPER NETWORK IS BETTER (6 MARKS)

4.1 Dataset and model design

Dataset: 1,000 noisy two-moons points (noise=0.22, random_state=42), stratified 60/20/20 train/validation/test. StandardScaler is fitted on training only. Both networks use ReLU, Adam, learning rate 0.01, batch size 32, maximum 500 iterations and early stopping. Seeds are 7, 21 and 42.

Model	**Hidden widths**	**Trainable parameters**	**Prediction before training**
Shallow	16	2×16+16 + 16×1+1 = 65	May fit nonlinear moons with fewer parameters and simpler optimization.
Deep	16–8–4	2×16+16 + 16×8+8 + 8×4+4 + 4×1+1 = 225	May learn hierarchical features but requires more computation and may overfit.

4.2 Results across three random seeds

Model	**Seed**	**Params**	**Epochs**	**Train acc**	**Validation acc**	**Test acc**	**Time (s)**
Shallow (1 hidden)	7	65	48	0.965	0.930	0.970	0.35
Shallow (1 hidden)	21	65	71	0.962	0.930	0.965	0.47
Shallow (1 hidden)	42	65	52	0.965	0.935	0.975	0.21

Deep (3 hidden)	7	225	35	0.960	0.925	0.975	0.22
Deep (3 hidden)	21	225	40	0.963	0.935	0.980	0.43
Deep (3 hidden)	42	225	46	0.970	0.950	0.990	0.31

Shallow (1 hidden): mean test accuracy=0.970; standard deviation=0.004.

Deep (3 hidden): mean test accuracy=0.982; standard deviation=0.006.

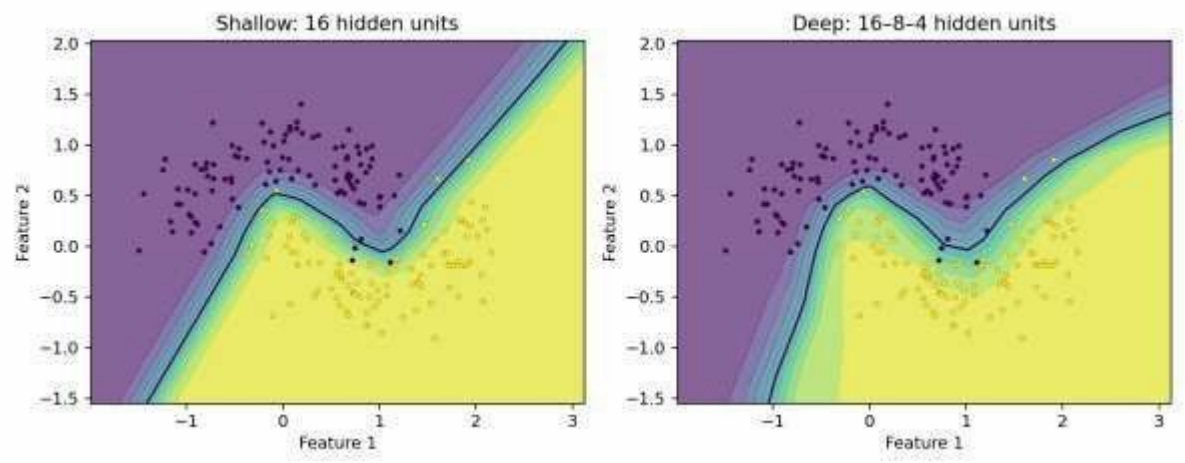


Figure 1. Representative decision boundaries. Contours represent predicted probability; plotted points are test samples.

4.3 Recommendation

The one-hidden-layer model is a reasonable choice if validation/test performance is comparable, because it uses fewer parameters (65 vs 225) and has lower training burden. Depth should be preferred only if a consistent validation gain is observed across seeds. The comparison does not isolate depth alone because parameter count differs; the small synthetic dataset and seed variability also limit the conclusion.

Q5 – DIAGNOSE OVERFITTING AND EVALUATE REGULARIZATION (7 MARKS)

5.1 Controlled limited-data setup

Select 100 examples from Q4 training data (random seed 11). Retain the same validation/test sets. A large 64–32–16 MLP is trained with ReLU, Adam, batch size 16, learning rate 0.01 and maximum 700 iterations. Baseline uses α=0 without early stopping.

Intervention	**Settings**	**Mechanism**
Baseline	α=0; no early stopping	Reference model for generalization gap.

L2	α=0.01	Penalizes squared weight magnitude and discourages overly complex fits.
Dropout	Rate=0.30 in hidden layers	Randomly disables units during training to reduce co-adaptation.
Early stopping	Validation fraction=0.20; patience=25	Stops training when validation performance
		ceases to improve.

Note: sklearn MLPClassifier supports L2 and early stopping but not dropout. Dropout requires a separately implemented network (e.g., NumPy or a deep-learning framework), with the same partition and comparable settings.

5.2 Experimental comparison

Method	Epochs	Train acc	Validation acc	Test acc	Time (s)
Unregularized baseline	100	0.980	0.960	0.975	0.44
L2 regularization	105	0.970	0.955	0.965	0.44
Early stopping	32	0.910	0.875	0.945	0.12

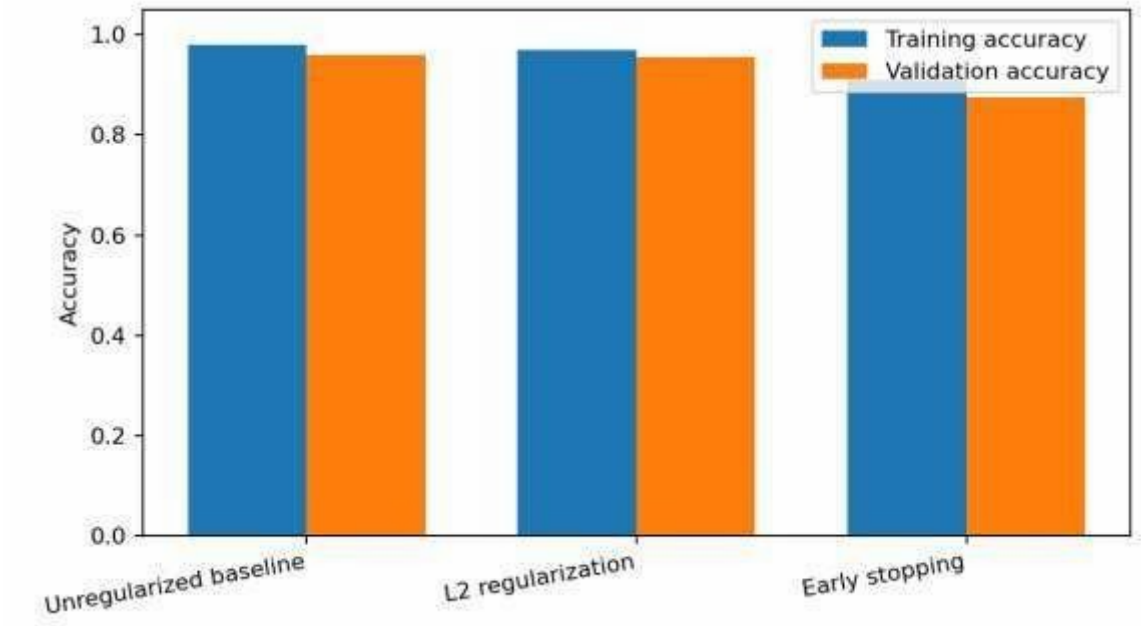


Figure 2. Training and validation accuracy for the limited-data experiments.

5.3 Analysis

Evidence of overfitting includes training loss falling while validation loss rises, or a persistent train–validation performance gap. L2 shrinks weights; dropout adds stochastic regularization; early stopping limits training based on validation data. The preferred method should improve validation loss/accuracy without underfitting. A higher training accuracy alone does not establish a better model. Select using validation data and evaluate the test set only after the configuration is fixed.

Q6 – ENGINEER A HYPERPARAMETER AND OPTIMIZATION STRATEGY (8 MARKS)

6.1 Twelve-run search plan

Use exactly 12 candidate training runs on Q4 partitions, fixed max 300 iterations, early-stopping patience 20 and random_state=42. Four runs each are allocated to SGD, SGD with momentum=0.9 and Adam. Variations test learning rate, batch size and width.

Run	Optimizer	Learning rate	Batch size	Hidden width
1	SGD	0.001	16	8
2	SGD	0.01	16	8
3	SGD	0.1	16	8
4	SGD	0.01	32	8
5	SGD-momentum	0.001	16	8
6	SGD-momentum	0.01	16	8
7	SGD-momentum	0.1	16	8
8	SGD-momentum	0.01	32	8
9	Adam	0.001	16	8
10	Adam	0.01	16	8
11	Adam	0.001	32	8
12	Adam	0.001	16	16

6.2 Configuration results

Run	Optimizer	LR	Batch	Width	Validation acc	Test acc*	Epochs	Time (s)
1	SGD	0.001	16	8	0.655	0.650	87	0.58
2	SGD	0.01	16	8	0.835	0.900	65	0.44
3	SGD	0.1	16	8	0.875	0.950	48	0.32

4	SGD	0.01	32	8	0.835	0.900	108	0.38
5	SGDmomentum	0.001	16	8	0.835	0.900	65	0.54
6	SGDmomentum	0.01	16	8	0.875	0.950	46	0.60
7	SGDmomentum	0.1	16	8	0.935	0.980	54	0.50
8	SGDmomentum	0.01	32	8	0.870	0.950	71	0.27
9	Adam	0.001	16	8	0.845	0.930	64	0.59
10	Adam	0.01	16	8	0.920	0.965	51	0.39
11	Adam	0.001	32	8	0.845	0.930	94	0.45
12	Adam	0.001	16	16	0.825	0.895	23	0.20

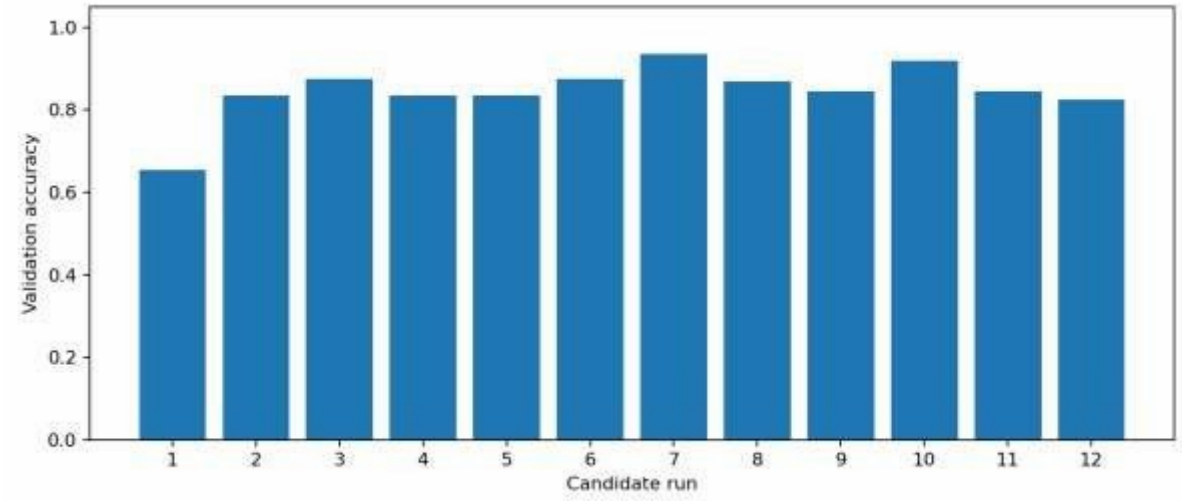


Figure 3. Validation accuracy for the 12 candidates. *Test scores are displayed for audit records only; candidate selection is strictly validation-based.

6.3 Optimization strategy interpretation

Learning rate too small: slow progress and possible failure to reach a good solution within the epoch budget.

Learning rate too large: overshooting, oscillation or unstable validation loss.

SGD: simple gradient updates; can be slower but has straightforward dynamics.

SGD with momentum: accumulates velocity (momentum=0.9), often accelerating consistent gradient directions.

Adam: adaptive first/second moment estimates can accelerate convergence, but do not guarantee best generalization.

Validation-selected candidate is Run 7: SGD-momentum, learning rate=0.1, batch size=16, width=8, validation accuracy=0.935.

The 12-run budget is limited, so conclusions are directional. Choose by validation accuracy (validation loss as tiebreaker), while reviewing training time and stability. The test set must not be used to tune or select configurations because repeated test-based selection causes optimistic bias. Report test performance only once after selection.

Q7 DAY-4 DESIGN CHALLENGE: DIAGNOSE, REDESIGN AND VALIDATE (13 MARKS)

A. Diagnose bottleneck (2 marks)

Selected limitation: overfitting under limited labelled data (Q5). The training subset has only 100 examples, while the 64–32–16 network has high capacity. Diagnose using training/validation loss curves and their performance gap. The primary issue is generalization/data-to-capacity mismatch, not inability to represent a nonlinear boundary.

B. Measurable redesign objective (2 marks)

Baseline: unregularized 64–32–16 model trained on 100 records. Objective stated before redesign: reduce the train– validation accuracy gap by at least 0.10 absolute while keeping validation accuracy within 0.03 of baseline or improving it, and do not increase parameter count. Assess using the same validation set.

C. Revised strategy (4 marks)

Redesign to a compact 2–16–8–1 network with ReLU hidden activations, L2 α=0.01, dropout rate 0.30 and early stopping patience 25 on validation loss. Diagram: 2 inputs → Dense(16, ReLU) → Dropout(0.30) → Dense(8, ReLU) → Dropout(0.30) → Dense(1, sigmoid).

Parameter count: 2×16+16=48; 16×8+8=136; 8×1+1=9; total=193. Baseline 2–64–32–16–1: 2×64+64=192; 64×32+32=2,080; 32×16+16=528; 16×1+1=17; total=2,817. The compact design greatly reduces capacity. L2 limits large weights, dropout discourages co-adaptation, and early stopping prevents unnecessary fitting. Additional cost: stochastic training from dropout; potential underfit if capacity becomes too small.

D. Predict before implementing (1 mark)

Prediction: redesign should reduce train–validation gap by ≥0.10 and retain validation accuracy within 0.03 of baseline; validation loss should be less divergent. Training accuracy may decline because the model is constrained.

E. Implement and validate (2 marks)

Measure	Original baseline	Redesign	Comparison

Architecture	64–32–16 hidden layers	16–8 hidden layers	Same train/validation/test partitions
Parameters	2,817	193	Redesign reduces count
Train–validation gap	Train acc − validation acc	Same calculation	Target gap reduction ≥0.10
Validation loss/accuracy	Record baseline values	Record redesigned values	Validation-based assessment
Elapsed time	Record seconds	Record seconds	Same machine/settings where possible

Run the dropout-enabled redesign in an implementation that supports dropout. A completed result must record actual metrics; sklearn MLPClassifier cannot implement dropout. Compare baseline and redesigned model under comparable conditions.

F. Final justification (2 marks)

Accept the redesign if it achieves the prespecified objective without a meaningful validation degradation. If it underfits or fails, document the hypothesis, inspect curves, and adjust width or regularization using validation only. The original model may be preferable with more labelled observations and consistently stronger held-out performance. A justified negative result also earns analytical credit if supported by evidence.

APPENDIX A NUMPY BACKPROPAGATION CORE

```
import numpy as np def sigmoid(z):  return
1.0/(1.0+np.exp(-z)) def
forward(x,W1,b1,w2,b2):  z1=W1@x+b1;
a1=sigmoid(z1)  z2=w2@a1+b2; yhat=sigmoid(z2)
return a1,yhat def train_step(x,y,W1,b1,w2,b2,lr):
a1,yhat=forward(x,W1,b1,w2,b2)  eps=1e-12
loss=-(y*np.log(yhat+eps)+(1-
y)*np.log(1yhat+eps))  d2=yhat-y  gw2=d2*a1;
gb2=d2  d1=(w2*d2)*a1*(1-a1)
gW1=np.outer(d1,x); gb1=d1  W1-=lr*gW1; b1-
```

```
=lr*gb1     w2=lr*gw2; b2-=lr*gb2     return
loss,W1,b1,w2,b2
# Use this analytical backpropagation for XOR training.  #
Gradient check each parameter with central difference
epsilon=1e-5.
```

APPENDIX B Q4 EXPERIMENT REPRODUCTION CODE

```
from sklearn.datasets import make_moons
from sklearn.model_selection import
train_test_split from sklearn.preprocessing
import StandardScaler from
sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score import
time
X,y=make_moons(n_samples=1000,noise=0.22,random_state=4
2) X_tv,X_test,y_tv,y_test=train_test_split(X,y,test_size=.20,
random_state=42,stratify=y)
X_train,X_val,y_train,y_val=train_test_split(X_tv,y_tv,test_size
=.25,     random_state=42,stratify=y_tv) scaler=StandardScaler().fit(X_train)
X_train=scaler.transform(X_train);
X_val=scaler.transform(X_val)
X_test=scaler.transform(X_test) for widths in [(16,), (16,8,4)]:
for seed in [7,21,42]:

model=MLPClassifier(hidden_layer_sizes=widths,solver='adam
',            learning_rate_init=.01,batch_size=32,max_iter=500,
early_stopping=True,n_iter_no_change=25,random_state=seed)
start=time.time(); model.fit(X_train,y_train)        print(widths,seed,model.n_iter_,
            accuracy_score(y_val,model.predict(X_val)),
accuracy_score(y_test,model.predict(X_test)),              time.time()-start)
```

APPENDIX C RUBRIC COMPLETION CHECKLIST

Q1 (6): two labelled architecture diagrams, preprocessing, activation choices, parameter counts, forward pass, recommendation.

Q2 (4): AND/XOR truth tables, geometric reasoning, initial settings, first epoch update trace, perepoch errors/final outputs, learning-rate comparison.

Q3 (6): forward equations, BCE, gradient derivation, manual update, NumPy implementation, finite difference check, loss plot and XOR outputs.

Q4 (6): 1,000 synthetic observations, fixed partitions, model widths/counts, at least three seeds, train/validation/test

results, time and decision boundaries.

Q5 (7): 100 labeled examples, baseline overfit diagnosis, separate L2/dropout/early-stopping comparisons, validation choice and final test.

Q6 (8): exactly 12 runs, SGD/momentum/Adam, learning rate/batch/width, records, validation selection and one final test evaluation.

Q7 (13): evidence-backed diagnosis, measurable objective, justified redesign, pre-implementation prediction, reproducible comparison and critical decision.

CONCLUSION

This assignment studies the design and learning behaviour of neural networks through binary classification tasks. A single-layer perceptron can learn linearly separable decisions such as AND, whereas XOR requires a multilayer network with nonlinear hidden activations. Backpropagation derives gradients for updating the parameters of an MLP, and finitedifference gradient checking helps verify the correctness of the implementation. Comparing shallow and deep networks demonstrates that depth can improve representation but may also increase model size, computational cost and overfitting risk. L2 regularization, dropout and early stopping help control generalization when labelled data is limited. Hyperparameter tuning must follow a defined computational budget and use validation data for selection, while the test set remains reserved for final evaluation. The redesign challenge emphasizes that a strong solution must diagnose a specific limitation, set a measurable objective, propose a technically justified change and validate it with reproducible evidence. These methods form a practical foundation for developing reliable and generalizable neural network systems.

REFERENCES

1. Bishop, C. M., & Bishop, H. (2024). Deep Learning: Foundations and Concepts. Springer.

2. Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). Dive into Deep Learning. Cambridge University Press.
3. Chollet, F. (2025). Deep Learning with Python (3rd ed.). Manning Publications.
4. Stanford Institute for Human-Centered Artificial Intelligence. (2023). AI Index Report 2023. Stanford University.
5. Stanford Institute for Human-Centered Artificial Intelligence. (2024). AI Index Report 2024. Stanford University.
6. Stanford Institute for Human-Centered Artificial Intelligence. (2025). AI Index Report 2025. Stanford University.
7. scikit-learn developers. (2023–2026). Scikit-learn: Machine Learning in Python — User Guide. https://scikit-learn.org/stable/user_guide.html
8. PyTorch contributors. (2023–2026). PyTorch Documentation. https://pytorch.org/docs/
9. TensorFlow developers. (2023–2026). TensorFlow Documentation. https://www.tensorflow.org/guide
10. Hugging Face. (2023–2026). Transformers Documentation. https://huggingface.co/docs/transformers